

Programa 2 explicado — a expressão (= 18)

Passo a passo de tudo que acontece no programa carregado na RAM: $(((7+3)-2)+5)-4+7-3+5 = 18$. Ele usa as 4 instruções (LDA, ADD, SUB, OUT) e mostra **dois** resultados: 9 no meio e 18 no fim.

Mapa da memória

end	binário	o que é	
0	0000 1011	LDA 11	PROGRAMA
1	0001 1100	ADD 12	
2	0010 1101	SUB 13	
3	0001 1110	ADD 14	
4	0010 1111	SUB 15	
5	1110 0000	OUT	
6	0001 1011	ADD 11	
7	0010 1100	SUB 12	
8	0001 1110	ADD 14	
9	1110 0000	OUT	
10	1111 0000	HLT	
11	0000 0111	= 7	DADOS
12	0000 0011	= 3	
13	0000 0010	= 2	
14	0000 0101	= 5	
15	0000 0100	= 4	

Como cada instrução roda (6 estados)

Toda instrução gasta 6 estados. Os 3 primeiros são **sempre iguais** (busca):

- **T1:** MAR $\leftarrow$ PC (pega o endereço da instrução)
- **T2:** PC $\leftarrow$ PC+1 (adianta o ponteiro)
- **T3:** IR $\leftarrow$ RAM (lê a instrução; o controlador passa a saber o opcode)

Os 3 últimos **dependem da instrução** (execução):

Instrução	T4	T5	T6
LDA n	MAR $\leftarrow$ n	A $\leftarrow$ RAM[n]	—
ADD n	MAR $\leftarrow$ n	B $\leftarrow$ RAM[n]	A $\leftarrow$ A+B
SUB n	MAR $\leftarrow$ n	B $\leftarrow$ RAM[n]	A $\leftarrow$ A-B
OUT	saída $\leftarrow$ A	—	—
HLT	congela	—	—

Trace completo — instrução por instrução

Estado inicial: **A=0, B=0, PC=0.**

#	PC	Instrução	O que faz	B	
1	0	LDA 11	$A \leftarrow \text{mem}[11]=7$	0	7
2	1	ADD 12	$B \leftarrow 3, A \leftarrow 7+3$	3	10
3	2	SUB 13	$B \leftarrow 2, A \leftarrow 10-2$	2	8
4	3	ADD 14	$B \leftarrow 5, A \leftarrow 8+5$	5	13
5	4	SUB 15	$B \leftarrow 4, A \leftarrow 13-4$	4	9
6	5	OUT	saída $\leftarrow A$	4	9 → mostra 09
7	6	ADD 11	$B \leftarrow 7, A \leftarrow 9+7$	7	16
8	7	SUB 12	$B \leftarrow 3, A \leftarrow 16-3$	3	13
9	8	ADD 14	$B \leftarrow 5, A \leftarrow 13+5$	5	18
10	9	OUT	saída $\leftarrow A$	5	18 → mostra 12 (hex de 18)
11	10	HLT	para em T4	5	18 (congelado)

A "história" do acumulador em duas metades: - **1ª metade** (até o 1º OUT): 7 → 10 → 8 → 13 → 9 = (((7+3)-2)+5)-4. - **2ª metade** (até o 2º OUT): continua de 9: → 16 → 13 → 18 = 9 + 7 - 3 + 5.

Os dois OUT

O OUT copia o acumulador **daquele momento** para o registrador de saída: - endereço 5: A=9 → mostra 09. - endereço 9: A=18 → mostra 12 (18 em hexadecimal).

Entre os dois, o visor **fica congelado em 9** — o registrador de saída só muda quando roda um OUT.

O HLT

No endereço 10, o HLT é detectado no **T4**, sobe hlt=1 e **congela o contador de estados em T4**. O clock continua, mas nada mais carrega.

O que ver na onda (do wave_sap1.do)

- **acc_out** (verde): a conta subindo/descendo — 7,10,8,13,9,16,13,18.
- **b_out**: muda a cada ADD/SUB (o operando: 3,2,5,4,7,3,5).
- **out_port** (laranja): pula pra 9, congela, depois pula pra 18.
- **opcode**: LDA, ADD, SUB, ADD, SUB, OUT, ADD, SUB, ADD, OUT, HLT.
- **hlt**: sobe no fim; **tstate** trava em T4.

Detalhes finos

1. **B é "descartável":** recarregado a cada ADD/SUB, não guarda resultado — é só o segundo número da conta do momento.
2. **A subtração liga Su=1 :** no T6 do SUB, além de `~La` e `Eu`, o `Su` ativa para a ALU subtrair.
3. **Dados são reusados:** `mem[11]=7` é lido na instrução 1 (LDA) e na 7 (ADD 11). O endereço é fixo; o dado fica lá para quem precisar.